

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 30

Annexure A
Dry Run Evaluation

Code of Conduct:

I G.Ramani (192312279) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

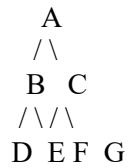
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

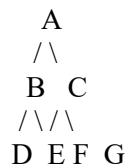
Iteration	Queue	Visited Node

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS)

Using the same graph,



Perform Depth-First Search (DFS) starting from A. Complete the table.

Iteration	Stack	Visited Node

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace Breadth-First Search (BFS) until the goal state is reached. Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem.

File Edit Format Run Options Window Help

Breadth-First Search (BFS) using Queue

```
from collections import deque
```

Graph Representation

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
```

BFS Function

```
def bfs(graph, start):
    visited = []
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print("Visited:", node)
            visited.append(node)

            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

            print("Queue:", list(queue))

    return visited

# Driver Code
result = bfs(graph, 'A')

print("\nBFS Traversal Order:")
print(" -> ".join(result))
```

File Edit Shell Debug Options Window Help

Python 3.14.4 (tags/v3.14.4:23116f9, A
AMD64)] on win32
Enter "help" below or click "Help" abo

>>>

===== RESTART: C:/Users/

```
Visited: A
Queue: ['B', 'C']
Visited: B
Queue: ['C', 'D', 'E']
Visited: C
Queue: ['D', 'E', 'F', 'G']
Visited: D
Queue: ['E', 'F', 'G']
Visited: E
Queue: ['F', 'G']
Visited: F
Queue: ['G']
Visited: G
Queue: []
```

BFS Traversal Order:

A -> B -> C -> D -> E -> F -> G

>>>

QUESTION-2

```
File Edit Format Run Options Window Help
# Depth-First Search (DFS) using Stack

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def dfs(graph, start):
    visited = []
    stack = [start]

    while stack:
        node = stack.pop()

        if node not in visited:
            print("Visited:", node)
            visited.append(node)

            # Push right child first so left child is visited first
            for neighbor in reversed(graph[node]):
                if neighbor not in visited:
                    stack.append(neighbor)

            print("Stack:", stack)

    return visited

# Driver Code
result = dfs(graph, 'A')

print("\nDFS Traversal Order:")
print(" -> ".join(result))
```

```
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, 7
AMD64)] on win32
Enter "help" below or click "Help" abc

>>>
===== RESTART: C:/Users/
Visited: A
Stack: ['C', 'B']
Visited: B
Stack: ['C', 'E', 'D']
Visited: D
Stack: ['C', 'E']
Visited: E
Stack: ['C']
Visited: C
Stack: ['G', 'F']
Visited: F
Stack: ['G']
Visited: G
Stack: []

DFS Traversal Order:
A -> B -> D -> E -> C -> F -> G

>>> |
```

QUESTION-

3

```

File Edit Format Run Options Window Help
from collections import deque

# Goal State
goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

# Initial State
start = [
    [1, 3, 6],
    [5, 0, 2],
    [4, 7, 8]
]

# Find the position of the blank (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

    for dx, dy in directions:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)

    return neighbors

# Breadth-First Search
def bfs(start, goal):
    queue = deque()
    queue.append((start, [start]))
    visited = set()

```

```

File Edit Shell Debug Options Window Help
Goal State Reached!
Number of Moves: 8

Solution Path:
Step 0
[1, 3, 6]
[5, 0, 2]
[4, 7, 8]

Step 1
[1, 3, 6]
[5, 2, 0]
[4, 7, 8]

Step 2
[1, 3, 0]
[5, 2, 6]
[4, 7, 8]

Step 3
[1, 0, 3]
[5, 2, 6]
[4, 7, 8]

Step 4
[1, 2, 3]
[5, 0, 6]
[4, 7, 8]

Step 5
[1, 2, 3]
[0, 5, 6]
[4, 7, 8]

Step 6
[1, 2, 3]
[4, 5, 6]
[0, 7, 8]

Step 7
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 8
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

```

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.